

Project Report: An Implementation of Stack-Level Website Fingerprinting Defenses

CS544: Topics in Networks

Team Members

Aditya Shukla (230101115)
Kartik Maheshwari (230101116)

Indian Institute of Technology Guwahati

Submitted to: Dr. T. Venkatesh
April 27, 2026

Abstract

Website Fingerprinting (WF) remains a significant privacy threat, allowing adversaries to identify encrypted web traffic by analyzing metadata patterns like packet sizes and timing. The 2025 HotNets paper, "Rethinking the Role of Network Stacks for Website Fingerprinting Defenses," argues that existing application-level defenses are ineffective because they are often "undone" by the operating system's network stack in its pursuit of efficiency. The paper proposes a framework, Stob, which moves obfuscation primitives directly into the kernel stack for robust enforcement.

This project successfully reproduces the core thesis of the Stob paper in a controlled environment using Mininet. We demonstrate the vulnerability of a standard Linux network stack, achieving 90.00% classification accuracy on our mock websites using a Random Forest (k-FP) attack. Subsequently, we implement Stob's core defense strategy by combining stack-level volume padding with timing jitter enforced via eBPF. Our cross-evaluation results show that this stack-level defense completely breaks the original traffic fingerprint, dramatically reducing the attacker's accuracy to a random baseline of 10.00%. When an adaptive attacker trains directly on the defended traffic, accuracy recovers to 30.00%. This report details our implementation, the challenges faced, and our key findings on the nuances of traffic analysis.

Contents

1	Introduction	3
2	Summary of the Original Paper	3
2.1	The Problem: The Application-Stack Gap	3
2.2	The Proposed Solution: Stob	4
3	Implementation Details	4
3.1	Environment and Dataset Generation	4
3.2	Feature Extraction and Attack Model	4
3.3	Implementing Stob Defenses	4
4	Results and Key Findings	5
4.1	Model 1: Baseline Vulnerability	5
4.2	Cross-Evaluation: Breaking the Fingerprint	5
4.3	Adaptive Attacker Evaluation	6
5	Challenges and Limitations	6
6	Conclusion	6
A	Code Repository	6

1 Introduction

The widespread adoption of encryption protocols like TLS has been a cornerstone of modern internet privacy, securing the content of our communications from eavesdroppers. However, a class of traffic analysis attacks known as Website Fingerprinting (WF) bypasses content encryption entirely. By applying machine learning techniques to traffic metadata—such as packet sizes, timing, and direction—an adversary can construct a “fingerprint” to identify which specific website a user is visiting with alarmingly high accuracy.

This threat is the central focus of the paper “Rethinking the Role of Network Stacks for Website Fingerprinting Defenses” by Lavrentieva et al. The authors identify a fundamental weakness in existing WF defenses: they are typically implemented at the application layer and are subsequently overridden by the underlying network stack’s performance optimizations. Their proposed solution, Stob, is a framework for integrating traffic obfuscation directly into the host operating system’s kernel.

Our project follows the “reproduction of empirical results” track for the CS544 course. Our goal was to validate the core claims of the Stob paper by building a simplified implementation. We set out to:

1. Demonstrate the vulnerability of a standard network stack in a controlled Mininet environment.
2. Implement Stob’s defense primitives—specifically Volume Padding and Timing Jitter—using modern kernel technologies like eBPF.
3. Quantify the effectiveness of this stack-level defense by performing cross-dataset evaluation to measure the drop in attacker accuracy.

2 Summary of the Original Paper

The Stob paper provides a compelling argument for a paradigm shift in how we design and deploy privacy-preserving network technologies. Its contributions are built upon identifying a critical conflict between application-level intentions and kernel-level realities.

2.1 The Problem: The Application-Stack Gap

The paper’s primary insight is what we term the “Application-Stack Gap.” When a privacy-conscious application attempts to hide a traffic fingerprint by adding artificial delays, the host’s network stack often reverses these efforts. As detailed in Section 2.3 of the paper, this happens due to:

- **Packet Coalescing:** To save CPU cycles and reduce header overhead, the kernel’s TCP implementation combines multiple small data writes from the application into single, large packets.
- **Asynchronous Buffering:** The kernel’s schedulers and congestion control algorithms determine the final departure time of packets, often ignoring the precise timing requested by an application.

The result is that the traffic pattern observed by a passive eavesdropper is the one optimized for efficiency by the stack, not the one designed for privacy by the application.

2.2 The Proposed Solution: Stob

To bridge this gap, the authors propose Stob (Stack-level Traffic Obfuscation), a framework implemented inside the network stack. By operating at the kernel level, Stob has the "final say" on packet sequences. Its architecture allows for the enforcement of key defense primitives like padding and timing modification, ensuring that obfuscation is physically represented on the wire.

3 Implementation Details

Our project was designed to be an end-to-end reproduction of the paper's core concepts. We used Mininet to create a controlled network, a Python-based server to generate fingerprints, and eBPF to implement the kernel-level defense.

3.1 Environment and Dataset Generation

We constructed a Mininet topology consisting of one client node and one server node connected via a virtual switch. A Python-based mock server was created to serve 10 synthetic "sites," each with a unique fingerprint defined by its requested file count and size. We utilized an automated collection script that fetched these sites multiple times, while `tcpdump` captured the traffic on the client's virtual interface, generating our *clean dataset*.

3.2 Feature Extraction and Attack Model

To implement the k-FP attacker, we used Scapy and tshark to parse the generated PCAP files. For each trace, we extracted seven vital features:

- **Volume Features:** `in_bytes`, `out_bytes`, `in_pkts`, `out_pkts`.
- **Timing Features:** Mean Inter-Arrival Time (`iat_mean`), IAT Range (`iat_range`), and IAT Median (`iat_median`).

These features were fed into a Scikit-learn `RandomForestClassifier` with 100 estimators.

3.3 Implementing Stob Defenses

Following the Stob architecture, we implemented a dual-pronged Regularization strategy acting strictly at the network stack level to obfuscate the traffic.

1. **Stack-Level Volume Padding:** Volume remains a highly deterministic side-channel. To neutralize it, we implemented stack-level padding that ensures every site transfer evaluates to a uniform target volume on the wire. This forces the `in_bytes` feature to be identical across all target sites.
2. **Timing Jitter (Stack-Level eBPF):** We wrote an eBPF program in C to inject a random delay of 0-5ms to every outgoing packet on the server. This program was compiled and attached to the Traffic Control (TC) egress hook, the very last point in the software stack before the hardware.

```
1 #include <linux/bpf.h>
2 #include <linux/pkt_cls.h>
3 #include <linux/types.h>
4 #include <bpf/bpf_helpers.h>
5
6 typedef __u64 u64;
7 typedef __u32 u32;
```

```

8
9 SEC("classifier")
10 int stob_defense(struct __sk_buff *skb) {
11     // Modify the packet's departure timestamp.
12     // The FQ (Fair Queuing) qdisc will hold the packet until
13     // this exact time, destroying the natural IAT timing fingerprint.
14     u64 now = bpf_ktime_get_ns();
15
16     // Add a base delay of 5ms + random jitter of 0-5ms.
17     u32 jitter = bpf_get_prandom_u32() % 5000000;
18     u64 delay = 5000000 + (u64)jitter;
19
20     skb->tstamp = now + delay;
21     return TC_ACT_OK;
22 }
23
24 char _license[] SEC("license") = "GPL";

```

Listing 1: The eBPF C code for injecting timing jitter (stob.kern.c).

4 Results and Key Findings

Our experiment produced clear, quantitative results that strongly support the thesis of the Stob paper. We evaluated the system under three different scenarios to map the effectiveness of the defense.

4.1 Model 1: Baseline Vulnerability

When trained and tested on the *clean dataset* (standard Linux stack behavior), our Random Forest model achieved **90.00% accuracy**.

Table 1: Classification Report for Model 1 (No Defense)

	precision	recall	f1-score	support
macro avg	0.94	0.92	0.91	10
weighted avg	0.93	0.90	0.89	10
accuracy			0.90	10

This high accuracy confirms that without any obfuscation, the host network stack leaks distinct structural characteristics (burst size, volume, pacing) directly onto the wire, making the attacker’s job highly reliable.

4.2 Cross-Evaluation: Breaking the Fingerprint

To test the core efficacy of the Stob framework, we took the model trained on the clean data and tested it against the *Stob-defended dataset*.

Table 2: Cross-Evaluation Results (Clean → Stob)

	precision	recall	f1-score	support
macro avg	0.01	0.10	0.02	50
weighted avg	0.01	0.10	0.02	50
accuracy			0.10	50

The accuracy plummeted to exactly **10.00%**. The combination of stack-level padding and eBPF timing jitter successfully obliterated the original traffic patterns, rendering the baseline model completely useless.

4.3 Adaptive Attacker Evaluation

A rigorous defense must withstand an attacker who is aware of the obfuscation strategy. We trained a new model (**Model 2**) exclusively on the Stob-defended dataset.

In this scenario, the adaptive attacker’s accuracy recovered to **30%**.

5 Challenges and Limitations

The implementation process involved several low-level networking challenges.

While volume padding and eBPF timing jitter were successfully implemented, altering the Maximum Segment Size (MSS) via standard `ip route` rules to enforce packet splitting did not work out as expected in our virtualized Mininet testbed. As a result, we focused the scope of our Stack-Level framework entirely on robust, highly deterministic volume padding and microsecond-level timing jitter. Additionally, ensuring that asynchronous `tcpdump` captures aligned perfectly with our automated Python scripts required significant synchronization logic to prevent data corruption across namespaces.

6 Conclusion

This project successfully reproduced the central empirical findings of "Rethinking the Role of Network Stacks for Website Fingerprinting Defenses." Our hands-on implementation in a controlled Mininet environment confirmed that a standard Linux network stack is highly vulnerable to WF attacks, yielding a 90.00% classification accuracy.

By implementing Stob’s Regularization strategy—specifically utilizing stack-level padding alongside eBPF-enforced timing jitter—we demonstrated the power of placing defenses directly in the kernel. This approach completely scrambled the traffic, reducing a baseline attacker’s accuracy to a random 10.00%. Even against an adaptive attacker, the defense successfully decreased identification accuracy by 60%. Our work highlights both the effectiveness of stack-level operations and the strict necessity of mitigating bidirectional leaks (both incoming payloads and outgoing requests) to ensure complete user privacy on the web.

References

- [1] Lavrentieva, E., Juarez, M., & Honda, M. (2025). Rethinking the Role of Network Stacks for Website Fingerprinting Defenses. In *Proceedings of the 24th ACM Workshop on Hot Topics in Networks (HotNets '25)*.

A Code Repository

All source code, eBPF scripts, and automated evaluation metrics associated with this project are available in our online repository.

https://github.com/kartik-2005/CS544_Course_Project